MZQA

SETUP GUIDE — FOR DUMMIES

Postgres Warehouse Setup

Standing up the database and pulling SEC, EDINET & Yahoo Finance data from a fresh clone

This repo ships the committee application — the FastAPI backend, the LangGraph tribunal, the Next.js frontend — and the *code* that builds the fundamentals warehouse. It does **not** ship the warehouse itself: no database dump, no CSVs, no pre-loaded Postgres instance. Every fact in `fact_fundamentals_us`, `fact_prices_intl`, and the other 190 tables has to be pulled from public sources — SEC EDGAR, Japan's EDINET, and Yahoo Finance — by running the CLI tools already in `backend/xbrl_sec/`. This volume is a start-to-finish, screenshot-free walkthrough for someone who has just cloned the repository and has an empty machine: which programs to install and where to get them, how to stand up Postgres, how to apply the schema, and the exact commands that kick off each data-acquisition run — with the quirks and gotchas the code itself doesn't warn you about.

Contents

1 What you are building 2
2 Before you start: tools to install 2
3 Step 1 — install and stand up Postgres 3
4 Step 2 — get the code and the Python/Node environments 3
5 Step 3 — environment variables 3
6 Step 4 — create the schema 4
7 Step 5 — pull SEC EDGAR data (US fundamentals) 5
8 Step 6 — pull EDINET data (Japan fundamentals) 6
9 Step 7 — pull Yahoo Finance data (prices & international equities) 6
10 Step 8 — the full from-scratch bootstrap, in order 7
11 Step 9 — verify it worked, then run the app 8
12 Reference: environment variables 8

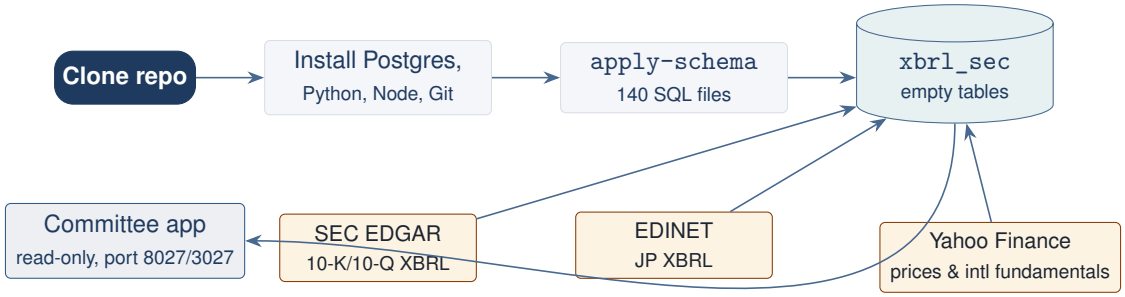

Figure 1. End-to-end picture: install tools → create schema → three independent ingestion pipelines fill the same warehouse → the application reads it. Nothing in this chain runs automatically; every step below is a command you type.

1 What you are building

Three independent pieces have to exist before the app shows real numbers: a running Postgres server, an empty `xbrl_sec` database with the schema applied, and that database filled in by three separate ingestion pipelines pulling from public data sources. The application itself (`backend/api/`) only ever *reads* — it has no write path, and no scheduler triggers any of this for you.

The one thing to internalize

There is **no** `docker-compose.yml`, **no** Makefile, **no** Airflow/Prefect DAG, and **no** scheduled job anywhere in this repo. Everything in this guide is a manual command run from a terminal, in the order given. If you skip a step, the next one will usually fail loudly (connection refused, table does not exist) rather than silently do the wrong thing.

2 Before you start: tools to install

Install these on a fresh Windows machine, in this order. Restart your terminal after each installer that touches environment variables (Git, Python, Node all do).

Tool	Why	Where to get it
Git	Clone the repository	`git-scm.com` — *Downloads for Windows*
PostgreSQL 16	The database server itself; the code assumes an already-running server and never installs one for you	`postgresql.org/download` → Windows → the EnterpriseDB installer (bundles `psql` and pgAdmin 4)
pgAdmin 4 (optional)	GUI to browse tables while you work; installed automatically by the EnterpriseDB installer above	bundled with the PostgreSQL installer, or standalone at `pgadmin.org`
Python 3.13 (64-bit)	Runs the ingestion CLIs and the FastAPI backend	`python.org/downloads` — tick *Add python.exe to PATH* during install
Node.js 18 or newer	Builds/runs the Next.js frontend	`nodejs.org` — the LTS installer

Why Python 3.13 specifically

The repo vendors a prebuilt `psycopg2` wheel for `cp313-win_amd64` (`backend/xbrl_sec/_vendor/psycopg2/`) as a fallback if `pip install` can't find a compatible binary. Any recent 3.11+ interpreter will run the code, but 3.13 matches what's already vendored and avoids a `pip` build-from-source step for psycopg2 on Windows.

3 Step 1 — install and stand up Postgres

1. Run the PostgreSQL installer. When prompted, set a password for the `postgres` superuser — write it down, you'll need it in Step 3.
2. Accept the default port (`5432`) and default locale unless you have a reason not to.
3. Leave Stack Builder's extra-component screen empty (nothing here needs PostGIS, etc.) and finish the install.
4. Open a terminal and confirm the server is reachable:

   ```
   psql -U postgres -h 127.0.0.1 -c "SELECT version();"
   ```

   It will prompt for the password you set in step 1. A version string back means the server is up.
5. Create the empty database the app expects (name and owner both matter — the code's defaults assume exactly this):

   ```
   psql -U postgres -h 127.0.0.1 -c "CREATE DATABASE xbrl_sec;"
   ```

Defaults baked into the code. Both connection paths in this repo default to `postgresql://postgres@127.0.0.1:5432/xbrl_sec` with schema `sec` (`backend/xbrl_sec/sec/settings.py`, `backend/api/settings.py`). If you used a different database name, port, or role when installing Postgres, you must override `DATABASE_URL` and `XBRL_SEC_DATABASE_URL` in `.env` to match — see Step 3.

4 Step 2 — get the code and the Python/Node environments

1. Clone and enter the repository:

   ```
   git clone <your-fork-or-remote-url> AI_Analyst
   cd AI_Analyst
   ```

2. Create a virtual environment and install the backend's *lean* dependency set (no torch/dash — that's `requirements-full.txt`, which this guide does not need):

   ```
   py -3 -m venv .venv
   .\.venv\Scripts\pip install -r backend\requirements.txt
   ```

3. Install the frontend dependencies (only needed once you want to see data in the browser — the ingestion CLIs in this guide don't touch the frontend):

   ```
   cd frontend
   npm install
   cd ..
   ```

No `pyproject.toml`, no Poetry, no uv
Dependency management here is plain `pip install -r requirements.txt`. There is also no SQLAlchemy and no Alembic anywhere in this codebase — the schema is raw SQL DDL (Section 6), not an ORM.

5 Step 3 — environment variables

Copy the template and fill in the values that matter to you:

```
copy .env.example .env
```

The Postgres *password* is deliberately not a line in `.env` at all — both connection paths (`psycopg2` and `asyncpg`) read it from the `PGPASSWORD` process environment variable at connect time. Set it once, permanently, from any terminal:

```
setx PGPASSWORD "the-password-you-set-in-step-1"
```

`setx` writes a *user* environment variable in Windows — open a new terminal window afterwards for it to take effect.

Variable	Purpose
`PGPASSWORD`	Postgres password, read directly by libpq — set with `setx`, not in `.env`.
`DATABASE_URL / DB_SCHEMA`	FastAPI backend's async (asyncpg) connection — default matches Step 1.
`XBRL_SEC_DATABASE_URL / XBRL_SEC_SCHEMA`	Ingestion layer's sync (psycopg2) connection — same default, override together with the two above if you changed the DB name.
`EDINET_API_KEY`	Required in practice for JP EDINET downloads — see Section 8.
`SEC_HTTP_USER_AGENT`	Read by the MD&A text-extraction module only — see the caveat in Section 7.
`DEEPSEEK_API_KEY (or another provider key)`	Only needed for LLM-scored steps: MD&A sentiment, 13F CUSIP matching, news scoring, and the committee itself. Not needed to just fill the fundamentals/price tables.
`MZQA_ROOT`	Defaults to a machine-specific path baked into `settings.py`; override it to this repo's root if a source throws `FileNotFoundError` pointing at a `...\MZQA` path.

6 Step 4 — create the schema

The entire warehouse schema is **140 sequential, idempotent SQL files** in `backend/xbrl_sec/sec/sql/` (`001_initial_schema.sql` through `135_quant_alpha_horizons.sql`). Every `CREATE TABLE/CREATE INDEX` is guarded with `IF NOT EXISTS`, so re-running the whole set against an already-current database is safe and is also how you pick up new migrations later.

```
$env:PYTHONPATH = "$PWD\backend"
.\.venv\Scripts\python -m xbrl_sec.sec.cli apply-schema
```

This runs each file in its own transaction and prints `applied <file>` or `FAILED <file>: <error>` per file — one failure does not abort the rest, so check the summary line at the end reads `failed=0`. It only creates structure; it loads no data.

What gets created

Two Postgres schemas: `sec` (everything — fundamentals, prices, 13F/insider, MD&A, macro, ETFs, quant) and `news` (articles, feeds, sentiment). 192 distinct tables in total. The groups that matter for the acquisition runs in Sections 5–7:

- **Run/pipeline metadata** — `pipeline_stage_run`, `pipeline_entity_state`, `source_filing_state`: per-entity watermarks so incremental runs know what's already been pulled.
- **Company/entity master** — `dim_company_us`, `dim_company_jp`, `dim_company_intl`, `ref_entity_ticker`: one row per known company, cross-referenced by CIK / EDINET

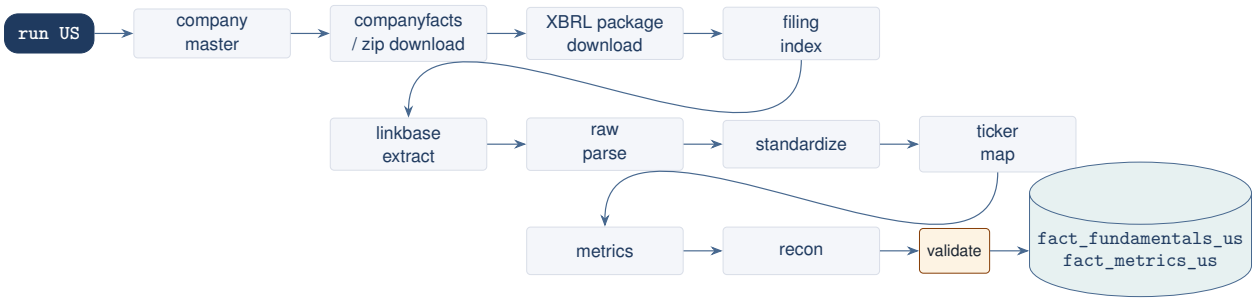

Figure 2. `run US -download` — the ordered stage chain (`backend/xbrl_sec/sec/pipelines/us.py`). Any stage can also be run alone (`download`, `extract`, `parse`, `metrics`, ...) to resume a partial run.

code / ticker.

- **Raw filing facts** — `fact_fundamentals_us`, `fact_fundamentals_jp`: the as-filed XBRL concept values, before any standardization.
- **Standardized financials** — `fact_fundamentals_std_us/_jp`, `fact_metrics_us/_jp`, `fact_metrics_recon_us/_jp`: governed line-items and derived metrics with source traceability.
- **International equities** — `fact_yahoo_fundamental_metric`, `fact_yahoo_statement_item`, `fact_metrics_intl`, `fact_prices_intl`, `ref_yahoo_index*`: the Yahoo-sourced non-US/JP universe.
- **Prices** — `fact_prices_us`, `fact_prices_jp`, `fact_market_metrics`, `fact_stock_split_event`.
- **13F / insider, MD&A, news, macro/factor, ETFs** — each acquisition pipeline in Sections 5–7 writes to its own subset; see the table beside each command block below.

7 Step 5 — pull SEC EDGAR data (US fundamentals)

```
$env:PYTHONPATH = "$PWD\backend"

# Everything, US, downloading what's missing
python -m xbrl_sec.sec.cli run US --download

# Just refresh the list of companies (new listings/delistings), no filings
python -m xbrl_sec.sec.cli refresh-master US --download

# Scope to specific companies by CIK instead of the whole universe
python -m xbrl_sec.sec.cli run US --entity 0000320193 --download
```

Writes to: `dim_company_us`, `fact_fundamentals_us`, `fact_fundamentals_std_us`, `fact_metrics_us`, `fact_metrics_recon_us`, `source_filing_state`, `ref_entity_ticker`.

No SEC account or API key needed — but read this before setting `SEC_HTTP_USER_AGENT`
SEC EDGAR requires every client to send an identifying `User-Agent` header (name + contact email) and self-throttle to roughly 10 requests/second — it does not require registration or an API key. The two modules that actually talk to EDGAR, `sec_download.py` and `sec_xbrl.py`, already satisfy this: they hardcode a compliant contact string and sleep between requests (roughly 8–9 req/s). **Setting `SEC_HTTP_USER_AGENT` in `.env` will not change what these two modules send** — it is only read by the separate MD&A HTML text-extraction module. If you want SEC to see *your own* contact address on these requests, edit the `_USER_AGENT` constant near the top of `sec_download.py` and `sec_xbrl.py`

directly.

8 Step 6 — pull EDINET data (Japan fundamentals)

1. **Get an EDINET API key.** EDINET's document/XBRL API (v2, base URL `api.edinet-fsa.go.jp/api/v2`) is public and free, but the current API generation expects a subscription key sent as the `Ocp-Apim-Subscription-Key` header. Register for one on the EDINET disclosure portal (`disclosure2.edinet-fsa.go.jp`) under its API user-registration section — this repo does not link the exact registration page, and the flow may be Japanese-language only.
2. Set the key once, permanently:

   ```
   setx EDINET_API_KEY "your-edinet-subscription-key"
   ```

   (On Windows the code also falls back to reading this from the registry `HKCU\Environment` if the process environment variable isn't set — `setx` writes exactly there, so either path works after a fresh terminal.)
3. Run the JP pipeline, same shape as US:

   ```
   python -m xbrl_sec.sec.cli run JP --download
   ```

Writes to: `dim_company_jp`, `fact_fundamentals_jp`, `fact_fundamentals_std_jp`, `fact_metrics_jp`, `fact_metrics_recon_jp`, `source_filing_state`, `ref_entity_ticker`.

The code does not hard-fail without a key
If `EDINET_API_KEY` is unset, the request-building code simply omits the subscription header rather than raising an error up front — EDINET's server will then reject or heavily rate-limit the calls itself, which shows up as widespread download failures partway through a run, not a clean error at startup. If a JP run is failing broadly, check the key first.

Filing type / document-code filter. `-filing-type` accepts EDINET document-type codes and is validated against a fixed set: `010, 020, 030, 040, 050, 120, 130, 140, 150, 160, 170`. An unrecognized code raises a `ValueError` listing the valid set — pass it (repeatable) to scope a run to, e.g., only annual securities reports.

9 Step 7 — pull Yahoo Finance data (prices & international equities)

Two independent Yahoo-based pipelines exist; neither needs an API key or registration — both use unauthenticated `yfinance` calls with a self-throttled, custom User-Agent.

7a. US price history and splits

```
# US-only price history (default fetches BOTH US and JP -- --skip-jp scopes to US)
python -m xbrl_sec.sec.cli fetch-prices --skip-jp

# Stock splits (jurisdiction is a flag here, defaulting to US)
python -m xbrl_sec.sec.cli fetch-stock-splits --jurisdiction US

# Derived market items (betas, etc.) -- no jurisdiction concept, always US
python -m xbrl_sec.sec.cli derive-market-items
```

A bare US positional argument will error
Older notes (including this repo's own `docs/data_pipeline.md`) show `fetch-prices US`, `fetch-stock-splits US`, `derive-market-items US`. None of the three commands accept a bare posi-

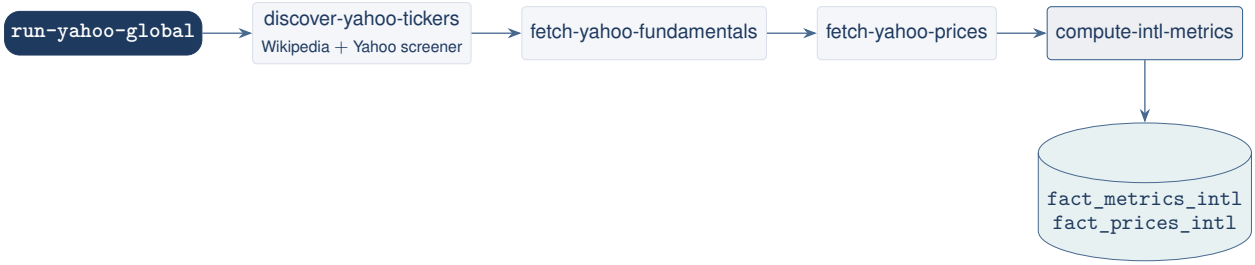

Figure 3. `run-yahoo-global` chains discovery → fundamentals → prices → metrics. Each stage is also an independent subcommand for resuming or re-running one piece.

> tional jurisdiction argument in the current `cli.py` — `fetch-prices` takes `-skip-us`/`-skip-jp` flags (default: both), `fetch-stock-splits` takes `-jurisdiction` (default `US`), and `derive-market-items` has no jurisdiction argument at all. Use the forms above.

7b. International equities (non-US/JP)

```
python -m xbrl_sec.sec.cli discover-yahoo-tickers    # -> dim_company_intl, ref_yahoo_index*
python -m xbrl_sec.sec.cli fetch-yahoo-fundamentals  # -> fact_yahoo_fundamental_metric,
    fact_yahoo_statement_item
python -m xbrl_sec.sec.cli fetch-yahoo-prices        # -> fact_prices_intl
python -m xbrl_sec.sec.cli compute-intl-metrics      # -> fact_metrics_intl
python -m xbrl_sec.sec.cli ingest-fx-intl            # -> fact_fx (or refresh-intl-fx for latest-
    only)
python -m xbrl_sec.sec.cli backfill-intl-gics        # no network -- fills GICS from stored
    strings
python -m xbrl_sec.sec.cli health-yahoo-global        # coverage / QA report

# Or the whole thing in one shot: discovery -> fundamentals -> prices -> metrics
python -m xbrl_sec.sec.cli run-yahoo-global
```

10 Step 8 — the full from-scratch bootstrap, in order

Putting Steps 4–7 together for a brand-new warehouse:

```
$env:PYTHONPATH = "$PWD\backend"

# 1. Schema
python -m xbrl_sec.sec.cli apply-schema

# 2. Reference / taxonomy tables the standardizers depend on
python -m xbrl_sec.sec.cli sync-refs
python -m xbrl_sec.sec.cli sync-registry
python -m xbrl_sec.sec.cli load-taxonomy-all-years

# 3. Core fundamentals -- slow, full SEC/EDINET history
python -m xbrl_sec.sec.cli run US --download --full
python -m xbrl_sec.sec.cli run JP --download --full

# 4. Prices
python -m xbrl_sec.sec.cli fetch-prices --skip-jp
python -m xbrl_sec.sec.cli fetch-stock-splits --jurisdiction US

# 5. International equities
python -m xbrl_sec.sec.cli run-yahoo-global

# 6. Everything else, as needed: mda, inst, insider, news, fetch-fred,
#    fetch-fama-french, compute-factor-model, cycle, etf.cli run
```

`-full` is slow
A `-full` US/JP run downloads and parses the entire filing history and can take hours. Day-to-day refreshes should drop `-full` and use plain `run <jurisdiction> -download` instead — it's incremental, driven by each source's watermark (`XBRL_SEC_US_LOOKBACK_DAYS` and similar settings in `backend/xbrl_sec/sec/settings.py`).

11 Step 9 — verify it worked, then run the app

```
# Coverage / row-count sanity checks
python -m xbrl_sec.sec.cli health-yahoo-global
python -m xbrl_sec.etf.cli status

# Token-free smoke test of the committee engine against the warehouse you just built
$env:PYTHONPATH = "$PWD\backend"
.\.venv\Scripts\python -m ai_analyst.committee.run --ticker MSFT --offline --no-news

# Start the app
.\start_committee.bat        # opens http://127.0.0.1:3027
```

If `start_committee.bat` shows an empty Explore view or a company with no financials, the most common causes are (in order): `apply-schema` never run, `PGPASSWORD` not set in the current terminal, or the relevant jurisdiction's `run <US|JP> -download` hasn't completed yet for that ticker.

12 Reference: environment variables

Variable	Used by
`PGPASSWORD`	Postgres auth (libpq), both connection paths — `setx`, not `.env`.
`DATABASE_URL / DB_SCHEMA`	FastAPI backend (asyncpg).
`XBRL_SEC_DATABASE_URL / XBRL_SEC_SCHEMA`	Ingestion CLIs (psycopg2).
`EDINET_API_KEY`	JP EDINET downloads (Section 8).
`SEC_HTTP_USER_AGENT`	MD&A HTML text extraction only — not the core SEC downloader (Section 7).
`FRED_API_KEY`	`fetch-fred` (macro data).
`OPENFIGI_API_KEY / OPEN_FIGI_API_KEY`	13F CUSIP → ticker matching.
`MOODYS_API_KEY, SP_API_KEY, FITCH_API_KEY`	ETF bond credit-quality ratings.
`DEEPSEEK_API_KEY (or OPENAI_/ANTHROPIC_/MOONSHOT_/GEMINI_API_KEY)`	MD&A scoring, 13F LLM matching, news sentiment, the committee itself.
`MZQA_NEWS_OLLAMA_URL`	News sentiment if using the local Ollama backend (default).
`MZQA_ROOT`	Machine-specific scratch/cache root; override if you see `FileNotFoundError` on another machine's path.

`MZQA_SKIP_SCHEMA` is a no-op
`.env.example` sets `MZQA_SKIP_SCHEMA=1`, inherited from the parent monorepo this app was carved out of. Nothing in this repository reads that variable — the FastAPI backend never calls `apply_schema()` regardless of its value. Schema management is always the manual `apply-schema` step in Section 6.